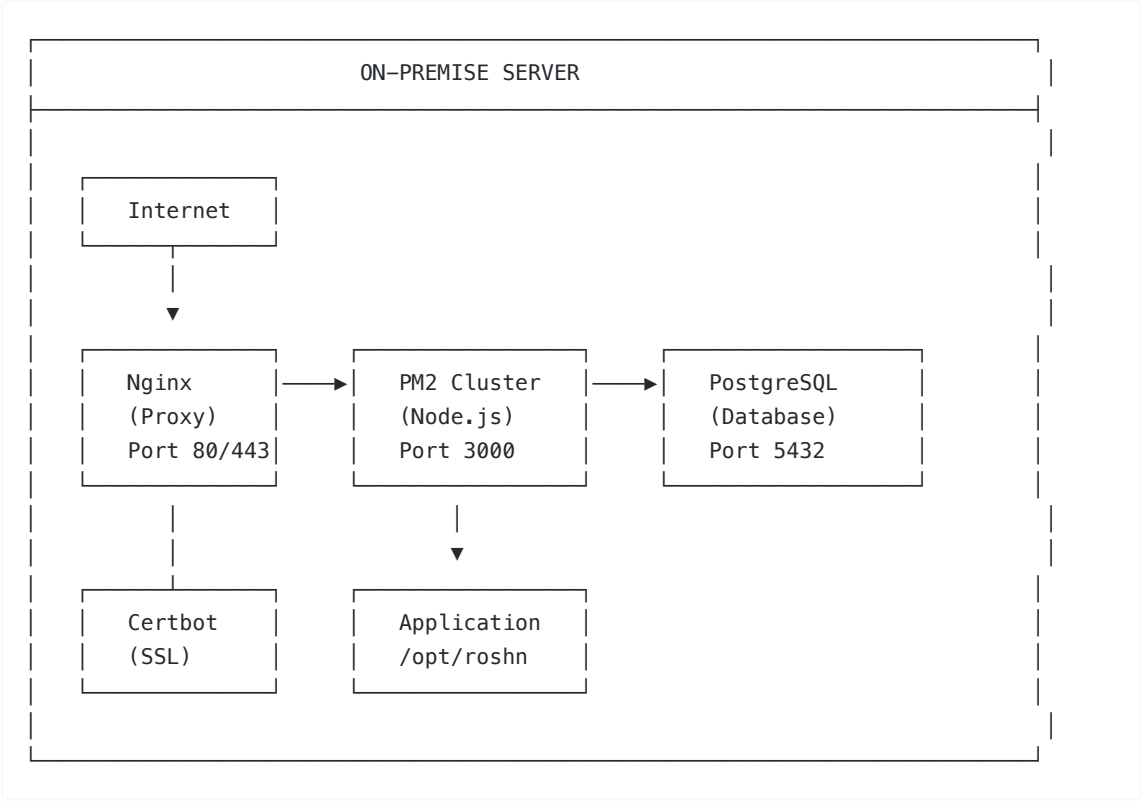


On-Premise Deployment Guide

Overview

This guide provides step-by-step instructions for deploying the ROSHN Parametric Masterplan Modelling Platform to an on-premise Linux server with PostgreSQL.

Architecture



Server Requirements

Minimum Specifications

Component	Development/Test	Production
Operating System	Ubuntu 22.04 LTS or RHEL 8+	Ubuntu 22.04 LTS or RHEL 8+
CPU	4 vCPUs	8 vCPUs
RAM	8 GB	16 GB
Disk Space	100 GB SSD	250 GB SSD
Network	100 Mbps	1 Gbps

Required Ports

Port	Service	Access
22	SSH	Internal only
80	HTTP	Public
443	HTTPS	Public
3000	Application	Internal only
5432	PostgreSQL	Internal only

Prerequisites

Server Access

- Root or sudo access to the server
- SSH access configured
- DNS configured for your domain

Domain Setup

Before running the deployment script, ensure:

- Domain points to your server's IP address
- DNS propagation is complete (check with `dig` or `nslookup`)

Deployment Steps

Step 1: Connect to Server

```
ssh root@your-server-ip
```

Step 2: Clone Repository

```
git clone https://github.com/tajalagawani/roshn.git
cd roshn
```

Step 3: Run Deployment Script

Run the deployment script with your target environment:

```
chmod +x scripts/deploy-onprem.sh

# Deploy to specific environment
sudo ./scripts/deploy-onprem.sh dev      # Development environment
sudo ./scripts/deploy-onprem.sh staging  # Staging environment
sudo ./scripts/deploy-onprem.sh prod     # Production environment (default)
```

Environment Configurations

Environment	Port	PM2 Instances	SSL	Domain
dev	3001	1	Optional	dev.platform.roshn.sa
staging	3002	2	Yes	staging.platform.roshn.sa
prod	3000	max (all CPUs)	Yes	platform.roshn.sa

Custom Configuration

You can also edit `scripts/deploy-onprem.sh` to customize:

Variable	Description	Example
DOMAIN	Your domain name	platform.roshn.sa
DB_PASSWORD	Database password (or leave empty to auto-generate)	
ENABLE_SSL	Enable Let's Encrypt SSL	true

What the Script Does

1. System Dependencies

Installs all required software:

- Node.js 20 LTS
- PostgreSQL 14+
- Nginx
- PM2 (process manager)
- Certbot (SSL certificates)
- Git

2. Database Setup

- Starts PostgreSQL service
- Creates database user
- Creates application database
- Grants necessary permissions

3. Application User

- Creates dedicated `roshn` system user
- Sets up home directory
- Configures permissions

4. Application Deployment

- Copies application to `/opt/roshn-platform`
- Installs npm dependencies
- Builds production application
- Creates environment configuration

5. Database Migrations

- Runs Prisma migrations

- Seeds initial data

6. Process Manager (PM2)

- Configures PM2 cluster mode
- Sets up automatic restart on failure
- Enables startup on boot
- Creates log directory

7. Nginx Configuration

- Sets up reverse proxy
- Configures WebSocket support
- Enables gzip compression
- Removes default site

8. SSL Certificate

- Obtains Let's Encrypt certificate
- Configures automatic renewal
- Redirects HTTP to HTTPS

9. Firewall

- Opens ports 22, 80, 443
- Blocks all other incoming traffic

10. Management Scripts

Creates convenient management commands in `/usr/local/bin/`

Post-Deployment

Access Your Application

Open your browser and navigate to:

```
https://your-domain.com
```

Verify Services

```
# Check application status
roshn-status

# View application logs
roshn-logs

# Check Nginx status
systemctl status nginx

# Check PostgreSQL status
systemctl status postgresql
```

Management Commands

The deployment script creates these convenience commands:

Command	Description
roshn-start	Start the application
roshn-stop	Stop the application
roshn-restart	Restart the application
roshn-status	Check application status
roshn-logs	View application logs

PM2 Commands

Additional PM2 commands for advanced management:

Command	Description
pm2 status	View all processes
pm2 logs	View all logs
pm2 logs roshn-platform	View application logs
pm2 monit	Real-time monitoring dashboard
pm2 reload roshn-platform	Zero-downtime reload
pm2 restart roshn-platform	Hard restart

Configuration Files

Application Directory

```
/opt/roshn-platform/  
├ .env                # Environment variables  
├ .next/              # Built application  
├ ecosystem.config.js # PM2 configuration  
├ node_modules/       # Dependencies  
├ package.json         # Project manifest  
├ prisma/              # Database schema  
└ public/              # Static assets
```

Log Files

Path	Description
------	-------------

/var/log/roshn/output.log	Application stdout
/var/log/roshn/error.log	Application stderr
/var/log/nginx/access.log	Nginx access logs
/var/log/nginx/error.log	Nginx error logs

Configuration Files

Path	Description
/opt/roshn-platform/.env	Application environment
/opt/roshn-platform/ecosystem.config.js	PM2 configuration
/etc/nginx/sites-available/roshn-platform	Nginx site config

Environment Variables

The `.env` file contains:

Variable	Description
DATABASE_URL	PostgreSQL connection string
NEXTAUTH_SECRET	Authentication encryption key
NEXTAUTH_URL	Application URL
NODE_ENV	Environment (production)

Updating the Application

Method 1: Pull and Rebuild

```
cd /opt/roshn-platform

# Pull latest code
sudo -u roshn git pull origin main

# Install any new dependencies
sudo -u roshn npm ci

# Rebuild application
sudo -u roshn npm run build

# Run any new migrations
sudo -u roshn npx prisma migrate deploy
```

```
# Restart application
roshn-restart
```

Method 2: Fresh Deployment

For major updates, re-run the deployment script:

```
sudo ./scripts/deploy-onprem.sh
```

SSL Certificate Management

Check Certificate Status

```
sudo certbot certificates
```

Renew Certificate Manually

```
sudo certbot renew
```

Auto-Renewal

Certbot automatically sets up a cron job for renewal. Verify with:

```
systemctl list-timers | grep certbot
```

Database Management

Connect to Database

```
sudo -u postgres psql -d roshn_platform
```

Backup Database

```
pg_dump -U roshn_admin -h localhost roshn_platform > backup.sql
```

Restore Database

```
psql -U roshn_admin -h localhost roshn_platform < backup.sql
```

Monitoring

Server Resources

```
# CPU and memory
htop

# Disk usage
df -h

# Process monitoring
pm2 monit
```

Application Health

```
# Check if application is responding
curl -I https://your-domain.com

# View real-time logs
pm2 logs roshn-platform --lines 100
```

Troubleshooting

Application Not Starting

1. Check PM2 logs:

```
pm2 logs roshn-platform
```

2. Verify environment file:

```
cat /opt/roshn-platform/.env
```

3. Check Node.js version:

```
node --version
```

Database Connection Errors

1. Check PostgreSQL is running:

```
systemctl status postgresql
```

2. Test connection:

```
psql -U roshn_admin -h localhost -d roshn_platform -c "SELECT 1;"
```

3. Check database URL in .env

Nginx Errors

1. Test configuration:

```
nginx -t
```

2. Check error logs:

```
tail -f /var/log/nginx/error.log
```

3. Restart Nginx:

```
systemctl restart nginx
```

SSL Certificate Issues

1. Verify domain DNS:

```
dig your-domain.com
```

2. Re-run Certbot:

```
sudo certbot --nginx -d your-domain.com
```

Security Recommendations

After Deployment

1. **Change default passwords** - Update database password if auto-generated
2. **Enable fail2ban** - Protect against brute-force attacks
3. **Regular updates** - Keep system packages updated
4. **Backup schedule** - Set up automated database backups
5. **Monitoring** - Configure alerting for downtime

Firewall Check

```
# Ubuntu/Debian
sudo ufw status

# RHEL/CentOS
sudo firewall-cmd --list-all
```

Backup Strategy

Daily Database Backup Script

Create `/opt/scripts/backup-db.sh` :

```
#!/bin/bash
BACKUP_DIR="/opt/backups"
DATE=$(date +%Y%m%d_%H%M%S)
mkdir -p $BACKUP_DIR
pg_dump -U roshn_admin roshn_platform > $BACKUP_DIR/roshn_$DATE.sql
# Keep only last 7 days
find $BACKUP_DIR -name "roshn_*.sql" -mtime +7 -delete
```

Schedule with Cron

```
# Edit crontab
crontab -e

# Add daily backup at 2 AM
0 2 * * * /opt/scripts/backup-db.sh
```

Support Information

Log Collection

When requesting support, collect:

```
# Application logs
pm2 logs roshn-platform --lines 500 > app-logs.txt

# System info
uname -a > system-info.txt
node --version >> system-info.txt
npm --version >> system-info.txt

# Nginx config
cat /etc/nginx/sites-available/roshn-platform > nginx-config.txt
```

Health Check Endpoints

Endpoint	Expected Response
/	200 OK
/api/health	200 OK (if configured)